

ISO 27001 Compliant Incident Management Report

SQL Injection Vulnerability

Introduction

This report details the identification and exploitation of an SQL injection vulnerability in the Damn Vulnerable Web Application. The test was conducted in a controlled environment to demonstrate a common vulnerability and its impact on data confidentiality.

Incident Description

An SQL injection flaw was discovered in the “User ID” input field of the SQL Injection module.

This vulnerability allows an attacker to manipulate the SQL query logic by injecting crafted payloads, thereby compromising the integrity of the database.

SQL Injection Method Used

Payload used in the User ID field to force the query to return all records:

```
' OR '1'='1
```

Evidence

- ID: 1' OR '1'='1 — First name: admin — Surname: admin
- ID: 1' OR '1'='1 — First name: Gordon — Surname: Brown
- ID: 1' OR '1'='1 — First name: Hack — Surname: Me
- ID: 1' OR '1'='1 — First name: Pablo — Surname: Picasso
- ID: 1' OR '1'='1 — First name: Bob — Surname: Smith

Incident Impact

- Confidentiality: High — Unauthorized disclosure of all user records.
- Integrity: Medium — Potential for data tampering.
- Availability: Low — No direct disruption of service.

Recommendations

1. Use Prepared Statements: Implement parameterized queries to prevent SQL injection.
2. Input Validation: Validate and sanitize all user-supplied data.
3. Deploy a WAF: Configure a Web Application Firewall to detect and block SQL injection patterns.
4. Regular Testing: Conduct periodic penetration tests and code reviews.

Conclusions

The SQL injection vulnerability in DVWA's User ID parameter highlights a critical lapse in input validation and query handling. Implementing the recommended controls will align the application with ISO/IEC 27001:2013 requirements and mitigate risks.